

Tarea 1: Inteligencia Artificial

501351-1

Autor: Vicente Ignacio Miranda Gómez

Profesor: Julio Godoy

Fecha: 28-09-2026

1. Introducción

En la siguiente tarea se plantea la evacuación de un grupo de personas representadas como Agentes, liderado por Sebastián, desde un piso de un edificio en llamas hacia la única salida de emergencias, antes de que el fuego se propague y los alcance. Este problema combina tres dificultades principales, el diseño del piso o mapa con obstáculos y pasillos angostos, la congestión, pues varias personas transitando por la misma casilla encarecen el costo de pasar por ahí, y el entorno que es dinámico, es decir, el fuego se propaga de forma irreversible y es necesario replanificar la ruta de escape.

El objetivo es implementar y comparar tres paradigmas de resolución. Búsqueda no informada (con algoritmos BFD y UCS), búsqueda informada guiada por heurísticas admisibles (Greedy y A*) y algoritmos genéticos, midiendo la tasa de supervivencia y el tiempo de despeje sobre tres mapas de distinta dificultad topológica.

2. Descripción del Problema y del Entorno

2.1 Representación del mapa

El mapa modela un piso representado como una grilla regular de filas x columnas. Cada casilla toma uno de cuatro valores: 0 representa libre, 1 un obstáculo, 2 la salida y 3 el fuego. La salida es única y está en la esquina inferior derecha. En los experimentos, los agentes parten agrupados cerca de la esquina izquierda, y el foco del incendio nace en una casilla libre elegida aleatoriamente.

2.2 Restricciones del Problema

- Congestión: En Entorno, obtener_costo asigna a cada casilla transitable un costo base de 1 más una penalización que crece con la ocupación de un número de agentes en la misma casilla. Se implementan tres funciones seleccionables por parámetro:
 - Lineal = $1 + 2n$
 - Cuadrática = $1 + n^2$
 - Exponencial = 2^n
- Propagación del Fuego: Cada K turnos el fuego se propaga a las casillas libres adyacentes de todas las casillas que están en llamas. La propagación es irreversible y el fuego no atraviesa muros ni alcanza la salida. Una casilla en llamas es infranqueable y un agente que es alcanzado por ella se considera una baja. Si la siguiente casilla de su ruta se incendia, debe replanificar. En este escenario se asume que existe un único foco de fuego, es decir, nace de una única casilla y se expande progresivamente.
- Movimiento Discreto: En cada turno un agente se desplaza una casilla en dirección ortogonal o permanece en su lugar con la acción “esperar”. BFS, UCS, Greedy y A* construyen rutas de movimientos. La acción esperar ocurre de forma implícita en cualquier algoritmo cuando el agente se queda sin ruta.

2.3 Mapas de Prueba

Existen tres mapas distintos y se generan colocando obstáculos al azar sobre casillas libres, en la proporción indicada:

- Mapa 1 (Alta densidad): Tiene un 70% de obstáculos, pasillos muy angostos que convergen hacia la salida y producen cuellos de botella.
- Mapa 2 (Densidad media): tiene un 30% de obstáculos, que pueden generar cruces y callejones sin salida.
- Mapa 3 (Baja densidad): Entorno semi abierto con múltiples rutas alternativas para llegar a la salida.

2.4 Dinámica de cada Turno

1. Se actualiza la ocupación (el número de agentes activos en cada casilla)
2. Cada agente vivo actúa: si su casilla está en llamas es una baja, si no, planificación cuando corresponde, avanza un paso de su ruta y verifica si ha llegado a la salida o cae en una llama de fuego.
3. Cada ciertos turnos el fuego se propaga

4. La simulación termina cuando no queden agentes en el mapa, ya sea porque los atrapó el fuego o porque lograron escapar, también puede terminar porque se llegó al límite de turnos totales.

3. Algoritmos Implementados

Los algoritmos comparten una lógica similar. Reciben el estado actual del entorno y la posición del agente, y devuelven una lista de casillas desde el siguiente paso hasta la salida. El agente conserva la ruta (con *ruta_planeada*) y la consume un paso por turno, solo vuelve a planificar cuando no tenga ruta o cuando la siguiente casilla quedó bloqueada.

3.1 Búsqueda en Anchura (BFS)

BFS explora las casillas por niveles usando una cola FIFO. Primero todas las casillas a un paso del agente, luego las de dos pasos y así sucesivamente. Se descartan las casillas con muro o fuego y las ya visitadas. Al alcanzar la salida retorna la ruta. Es un algoritmo completo y, como todos los pasos se consideran de igual costo, entrega la ruta con menor número de movimientos. Su limitación es que se ignora la congestión, ya que una casilla saturada le parece igual que una vacía. Su complejidad es $O(V + E)$, es decir, lineal en el número de casillas.

3.2 Búsqueda de Costo Uniforme (UCS)

UCS reemplaza la cola FIFO por una cola de prioridad ordenada por el costo acumulado $g(n)$, calculado en *obtener_costo*. Siempre se expande la casilla de menor costo acumulado, por lo que la primera vez que extrae la salida ya encontró la ruta de menor costo total, incorporando la penalización por congestión. Se descartan las entradas obsoletas de la cola y se usa un contador para desempatar entrada de igual costo. Es completo y óptimo en costo, pero explora en todas las direcciones sin ninguna noción de dónde está la salida.

3.3 Distancia Manhattan

Se utiliza la distancia Manhattan como heurística admisible, los algoritmos informados utilizan:

$$h(n) = |fila(n) - fila(salida)| + |col(n) - col(salida)|.$$

Se dice que es admisible ya que nunca sobreestima el costo real restante.

- Como los movimientos son ortogonales, cada paso cambia la distancia Manhattan exactamente en una unidad, por lo que se necesitan al menos $h(n)$ pasos para llegar a la salida.
- Cada paso cuesta al menos 1, porque el costo es 1 más penalización positiva.

3.4 Búsqueda Voraz (Greedy Best-First)

Greedy ordena la frontera guiándose únicamente por $h(n)$. Siempre expande la casilla que parece más cercana a la salida. No considera el costo que se ha recorrido ni la congestión, esto lo hace la más rápida de calcular, pero no garantiza rutas óptimas y puede internarse en callejones sin salida que reducen la distancia Manhattan.

3.5 A estrella (A^*)

Este algoritmo ordena la frontera por:

$$f(n) = g(n) + h(n)$$

$g(n)$ es el costo acumulado y $h(n)$ es la distancia Manhattan.

Con una heurística admisible y consistente, A* entrega una ruta con el mismo costo óptimo que la de UCS pero expandiendo menos casillas, ya que la heurística orienta la búsqueda hacia la salida.

3.6 Algoritmo Genético

El algoritmo genético es una metaheurística bioinspirada, es decir, inspirada en la evolución biológica y la selección natural para resolver problemas complejos. Aborda la planificación como un problema de optimización. Sus componentes son:

- **Representación:** Cada individuo es un cromosoma, es decir, una secuencia de genes, en este caso cada uno es una acción del conjunto (arriba, abajo, izquierda, derecha, esperar).
- **Decodificación:** El cromosoma se ejecuta gen a gen sobre el estado actual del entorno, desde la posición del agente. Un movimiento contra un muro, el fuego o el borde no avanza y se cuenta como un movimiento inválido. “Esperar” mantiene la posición y consume un turno. La ejecución se detiene al encontrar la salida y el resultado es una ruta de casillas del mismo tipo de los demás algoritmos.
- **Fitness Function:**
$$\text{costo} = \text{costo congestión acumulado} + \text{pasos utilizados} + 25 \cdot \text{movimientos inválidos}$$

Si el individuo no llega a la salida se suma además una penalización de 10 mil más 10 veces la distancia Manhattan restante. Esta penalización grande asegura que cualquier individuo que llegue supere a cualquiera que no llegue.
- **Inicialización:** La población inicial se genera con caminatas sesgadas. En cada paso con una probabilidad de 0.6 se elige (si es que existe) una dirección válida que reduce la distancia Manhattan a la salida. En pruebas preliminares con inicialización completamente aleatoria y fuera del experimento reportado, se presentó una supervivencia bastante baja, entre 9% a 50% según el mapa, esto quiere decir que una población aleatoria casi nunca contiene subsecuencias dirigidas a la salida. Por eso se adopta la inicialización sesgada y se aumenta la población y generaciones.
- **Operadores:** Selección por torneo con 3 participantes, cruce de un punto (probabilidad 0.85), mutación por gen (probabilidad de 0.15) y elitismo (el mejor individuo pasa intacto a la siguiente generación).
- **Integración con la replanificación:** El algoritmo genético puede no alcanzar la salida en una sola ejecución. En ese caso, el agente sigue la mejor ruta parcial encontrada y al agotarla se ejecuta nuevamente el algoritmo desde su nueva posición. Como el costo de congestión se evalúa sobre la ocupación del instante de planificación, el fitness de la práctica no premia esperar.

El ciclo evolutivo se puede resumir en los siguientes pasos:

1. Crear la población inicial con caminatas sesgadas hacia la salida
2. Decodificar cada individuo sobre el estado actual del entorno y calcular su fitness
3. Guardar el mejor individuo. En el caso de que alcanza la salida con longitud mínima y sin movimientos inválidos, terminar
4. Formar la nueva generación, es decir, copiar el mejor individuo y completar con hijos obtenidos por selección por torneo, cruce de un punto y mutación
5. Repetir desde el paso 2
6. Decodificar el mejor individuo y entregar su ruta al agente

4. Experimentación

4.1 Diseño del experimento

Cada combinación de mapa y algoritmo se ejecutó 80 veces, lo que da un total de 1200 soluciones (3 mapas x 5 algoritmos x 80 iteraciones). En la siguiente tabla se pueden ver los parámetros del experimento.

Parámetro	Valor
Tamaño de la grilla	20 x 20 casillas
Salida	Esquina inferior derecha (19,19)
Agentes por simulación	12 en casillas distintas elegidas al azar dentro de una región de 6x6 en la esquina superior izquierda
Foco del incendio	Casilla libre al azar (respetando salida y agentes)
Propagación del fuego	Cada $k = 4$ turnos
Costo congestión	Cuadrática
Límite de turnos	300 por simulación
Densidad de obstáculos	Mapa 1: 70%, Mapa 2: 30%, Mapa 3: 10%

4.2 Métricas

- Tasa de Supervivencia: Cociente entre los agentes que alcanzan la salida y el total de agentes, agregados sobre las 80 iteraciones. Se acompaña de un intervalo de confianza (IC) del 95% por bootstrap con 5 mil remuestreos, tomando como unidad de remuestreo la corrida completa, porque los agentes de una misma corrida no son independientes.
- Tiempo de despeje: Número de turnos que tomó al último sobreviviente en alcanzar la salida, esto es el máximo de los turnos empleados por los agentes que escaparon en esa corrida. Se reportan media, desviación estandar, mínimo y máximo sobre las corridas con al menos un sobreviviente. Las que no tienen sobrevivientes se informan por separado.
- Métricas complementarias: Indicadores del costo computacional para estudiar los tiempos de CPU por simulación

5. Resultados

Mapa 1 (Alta densidad)

Algoritmo	Supervivencia	Sin sobrevivientes	Media	Desv. estandar
BFS	82.8%	9/80	38.0	2.8
UCS	82.7%	9/80	38.0	2.8
Greedy	82.8%	9/80	38.0	2.8
A*	82.7%	9/80	38.0	2.8
Genético	70.7%	14/80	74.5	40.3

Mapa 2 (Media densidad)

Algoritmo	Supervivencia	Sin sobrevivientes	Media	Desv. estandar
BFS	78.4%	9/80	39.1	5.1
UCS	79.8%	8/80	39.8	5.6
Greedy	74.7%	11/80	42.6	7.5
A*	79.8%	8/80	39.7	5.4
Genético	47.0%	20/80	70.7	26.6

Mapa 3 (Baja densidad)

Algoritmo	Supervivencia	Sin sobrevivientes	Media	Desv. estandar
BFS	68.2%	15/80	36.2	2.5
UCS	71.9%	13/80	36.6	2.0
Greedy	65.4%	15/80	38.4	3.7
A*	71.5%	13/80	36.6	1.9
Genético	58.8%	17/80	51.4	7.3

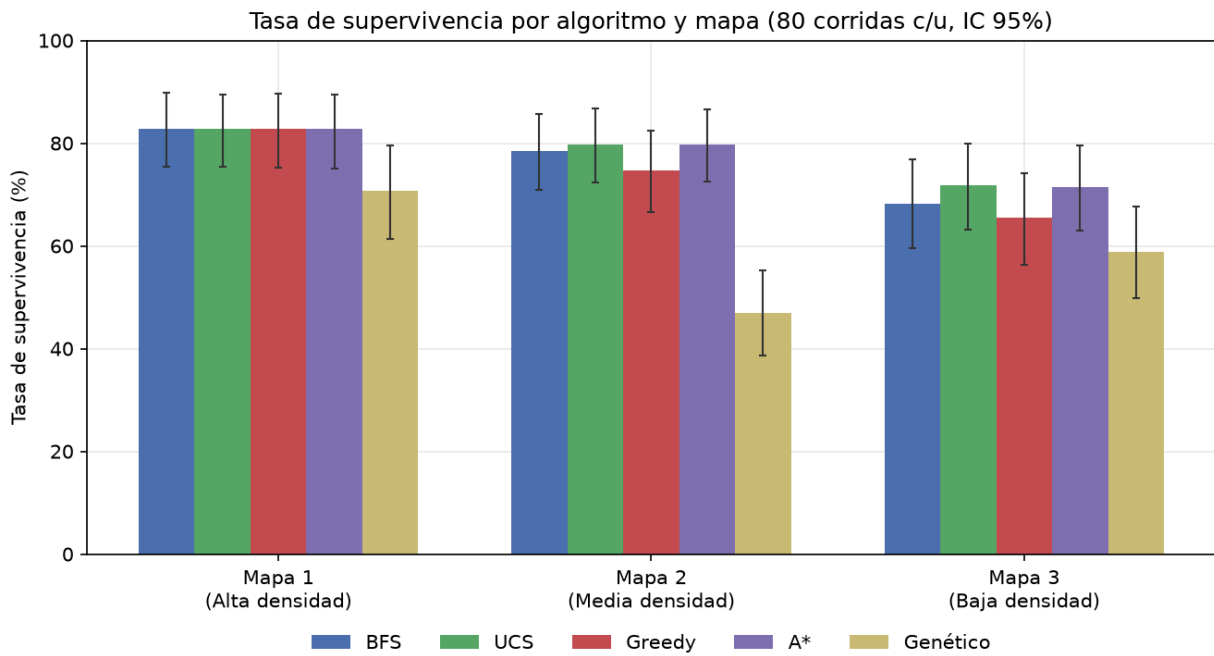


Imagen 1. Tasa de supervivencia por algoritmo y mapa. Las barras de error son IC 95% bootstrap.

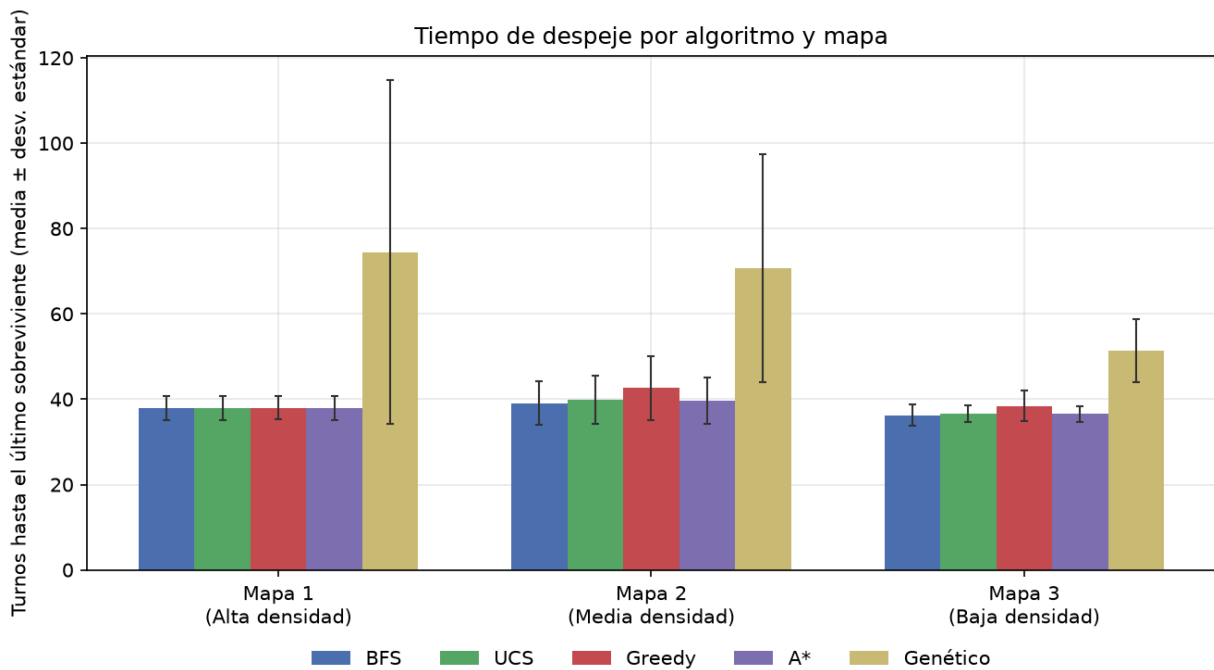


Imagen 2. Tiempo de despeje (turnos hasta el último sobreviviente)

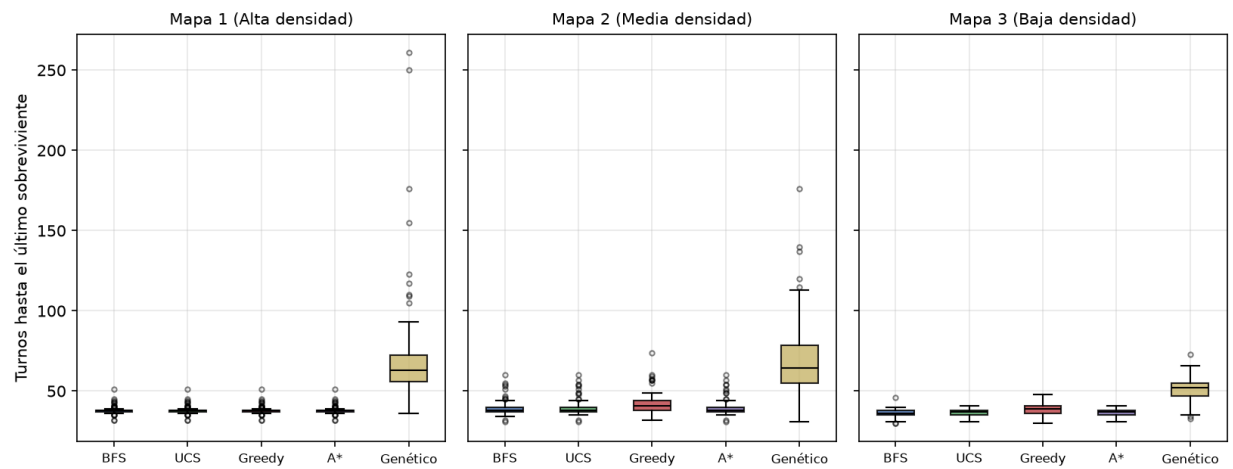


Imagen 3. Distribución del tiempo de despeje por algoritmo y mapa

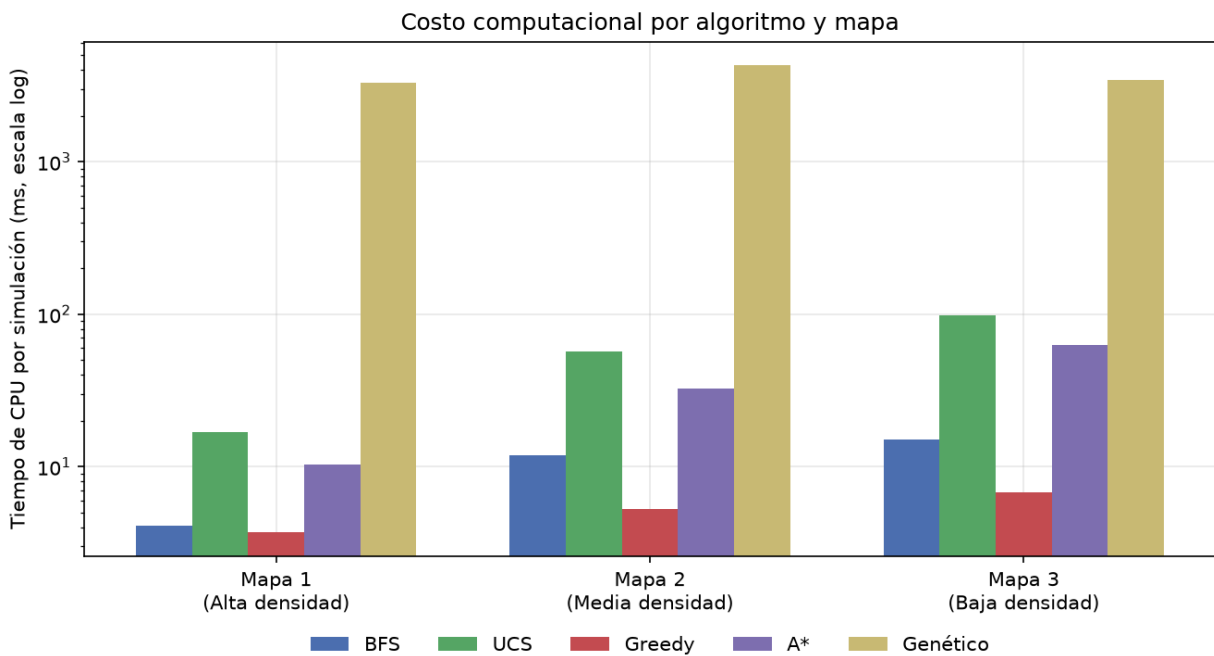


Imagen 4. Costo computacional

6. Análisis y Conclusiones

6.1 Efecto de la topología del mapa

Para BFS, UCS, Greedy y A* la supervivencia disminuye a medida que baja la intensidad de obstáculos. Con BFS por ejemplo pasa de 82.8% en el mapa 1 a 78.4% en el mapa 2 y 68.2% en el mapa 3. De primeras parece contradictorio, ya que un mapa abierto ofrece más rutas alternativas. Una hipótesis plausible, es que en este modelo los obstáculos también frenan el fuego (se pueden imaginar como grandes muros de metal) y solo se propaga a casillas libres, mientras que el abierto el fuego avanza como un frente amplio que alcanza más rutas y oportunamente a más agentes. Esto se podría comprobar midiendo la superficie quemada por turno en cada mapa, algo que no está implementado en esta entrega.

En el Mapa 1 los cuatro algoritmos (sin contar al genético) se comportan de forma idéntica. Esto es coherente con que los pasillos angostos dejen prácticamente una sola ruta viable. Cuando no hay alternativas, la heurística o el costo de congestión no tiene qué elegir.

6.2 Algoritmo Genético

Éste obtuvo la menor supervivencia en los tres mapas, también necesitando más turnos y con mayor dispersión. Aún así, su tiempo mínimo es comparable al de los demás. Cuando converge encuentra rutas casi óptimas, teniendo el problema de la consistencia.

6.3 Conclusiones

Se implementaron y compararon cinco algoritmos de tres paradigmas distintos sobre 1200 simulaciones. Los métodos de búsqueda con costo de congestión, fueron los de mejor supervivencia en los mapas con rutas alternativas. A* logró la misma calidad que UCS pero con menos CPU (entre 31% a 42% menos), por lo que es la opción más recomendable para este problema.

Por otra parte BFS es una base sólida y barata, mientras greedy es la más rápida pero produce rutas mas largas y pierde supervivencia en los mapas con alternativas.

La topología condicionó el resultado más que la elección del algoritmo de búsqueda. En el Mapa 1 los cuatro coinciden, y entre 10% y 15% de las instancias resultaron fatales para todos los algoritmos por lo se puede concluir que el techo de supervivencia lo fija la ubicación en la que nace el incendio y no el método de planificación.